CS3217 Project Proposal: TowerForge

Team Members
Keith Gan – [redacted]
Phua Zheng Ze – [redacted]
Rubesh Suresh – [redacted]
Vanessa Mae - [redacted]

TowerForge Overview

We plan to develop a grid based tower defense builder. Players should be able to create their own levels with a level builder as well as specify the rules that govern the game world such as positions where towers can be placed and the like. We hope to allow players to be able to make relatively simple customisations within the application and provide an easy to extend source code for players to further customize the game to their vision.

Basic Application Usage

In the level designer, players can specify parameters such as path arrangements, enemy types, enemy counts, minion timer and victory conditions, etc. Players will be given the ability to design the specific rules according to which the game will be played. This way, the players will have maximum control over the exact kind of game experience that they want to have.

Gameplay

During the game itself, players can place units (minions) to **attack**. Players can decide where they want minions in order to attack the opponent player and hence reduce their Health Points (HP). The player whose tower HP reaches zero first will lose the game.

Tower

Players have a tower which is the game entity that they need to defend

- Once minions manage to enter the tower, the HP point of the player will be reduced.

Minions / Units

Players will have their own units that they can use to attack the opponents.

- There will be a timer in each minion slot, once it has finished loading up, players can then choose which path lane they want to place their minions.
- The minions will start from the starting point.
- Players need to pick the specific grid at which they want to place the minions.
- The minions can attack the opponent's minions and each minion has their own **health.** Once the health has reached 0, they will disappear and the winning minion can proceed with their movement towards the tower.
- The minion also has a **speed** component that will determine how fast it moves across the board

Combat Mechanics:

- Minions/units are capable of engaging in combat with enemy counterparts or directly attacking enemy towers.
- Each minion/unit possesses its own set of attack parameters, such as damage dealt per attack and attack speed

Existing Applications vs. TowerForge

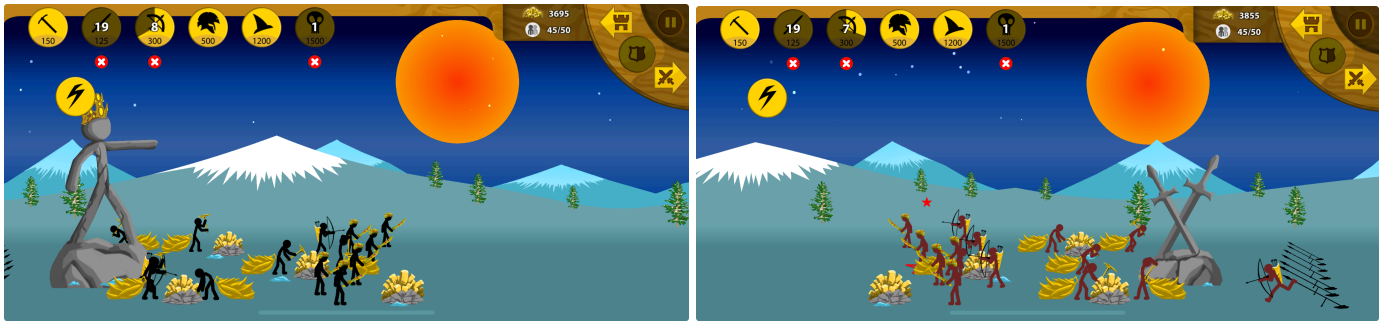
Similar Applications

There are quite a few tower defense games such as PvZ, StickWar and Bloons TD Battles. While there are many tower defense games available, our application combines the classic tower defense strategy with the charming and strategic elements of PvZ. Players will not only focus on effective tower placement but also strategically choose and upgrade towers with unique abilities to fend off waves of enemies. This hybrid approach sets our game apart from traditional tower defense titles. However, these games usually have a fixed set of playable units and players have no ability to customize the game and have to make do with the selection provided by the game. We aim to retain the set of features that make these types of game enjoyable whilst adding more features in the areas where these games lack. We are designing a level-builder for the game so that the user can define a game with their own set of rules, towers, enemies, enemy behavior. Users can customize the number of towers, enemies, and paths in the game so players can unleash their creativity and design their own unique gaming experiences. Users can also customize the allowable objects in each tile of the grid. The game will also support multiplayer and single-player so that a variety of games can be designed.

What makes our app special

We took inspiration from Warlords: Call to Arms as well as Stick War: Legacy where units are sent to destroy the enemy's base.

For example, shown below is the gameplay for Stick War: Legacy. The tower on each end of the screen represents the player's base and what they are to protect. Each player can generate running (dynamic) units that they can send to attack the player on the other side (similar to the game Age of War as well).



The player earns gold by creating miners who mine for gold, and this gold can be spent to create more units that can fight. However, that's the limit to strategizing in the game. Once the game begins, all units are dynamic and there is not much strategy required.

This contrasts with Plants vs Zombies on the right which follows closer to the classic tower defense style where units are stationary (static) and once the game begins, while more units can be added, all units are static and the scope of strategizing is limited.



We aim to combine the static and dynamic styles of both game types and infuse that into TowerForge, maximizing the scope of strategizing by extending it to both during level designing and during gameplay.

Features Overview

Main Features

1. **Theme customisation**
    a. Terrain manipulation : introducing a feature for the players to customize the terrain of the battlefield. This can include changing the map of the level.
2. **Grid customisation**
    a. Users should be able to specify the dimensions for the grid where towers can be placed.
    b. Users should be able to edit the rules of any sub-grid.
        i. For example, customizing elevation of the tile in the grid, preventing enemy units from being able to reach the tower directly.
3. **Enemy path specification**
    a. Users should be able to customize the number of paths as well as the paths taken by the enemies(e.g. in a zigzag manner)
4. **Unit customisation**
    a. Visual customisation: Just like for towers, this feature will allow players to customize the appearance of the units.
    b. Functional Customisation: Customizing the behavior of the units. For example, in addition to being able to customize the stats, we can change the behavior of the enemies to target the towers or to rush to damage the player.

Possible extensions:

Game Extensions (Gameplay and User experience focused)

1. Custom tower upgrades
    a. Users will be able to define the upgrade path of the towers and tweak the stats and cost for each the upgrades
2. Power Spell
    a. Players can summon magic spells that will have special effects towards the minions that they applied the spells to
3. Unit placement
    a. Units do not have to be constrained to being positioned at the start of the map - this can be extended to a region at the start
4. Tower Customisation and Placement
    a. Visual customisation : Allow players to customize the appearance of the towers like changing the colors, adding extra cosmetic features to it. This adds a layer of personalization for the users
    b. Functional Customisation : Customizing the behavior of the towers.
        i. For example, customizing the attack and health power of the towers as well as toggling behavior options such as targeting the nearest unit or targeting the unit with the highest health.

Functionality Extensions (Software Engineering focused)

1. Multiplayer
    a. Players will be able to play against one another using one of the levels that they have designed.
2. Collaborative Level Design
    a. Players will be able to team up and design a level in real time, collaboratively.

Project Schedule Overview

Project Milestones

Project proposal	Sunday, 10 March 2024
Sprint 1	Sunday, 24 March 2024
Sprint 2	Sunday, 7 April 2024
STePS	Wednesday, 17 April 2024
Final project report	Sunday, 21 April 2024
Final project presentation	Tuesday, 23 April to Friday, 26 April 2024

Project Schedule

Sprint	Week	Date	Task
-	Week 7	March 3	Admin Tasks - Set up Workflow, CI/CD, etc
1	Week 8	March 10	Models (Grid, Game Entities, Persistence) + Unit test
	Week 9	March 17	Game Loop, Event Queue, Game Engine, UI + Controllers
2	Week 10	March 24	Behaviour Customisation (Types, Paths, Enemy Prioritisation)
	Week 11	March 31	Multiplayer Network Adapter* (or other extensions)
3	Week 12	April 7	Multiplayer Network Adapter* (or other extensions)
	Week 13	April 14	Finalize project and Bug Fixing

Sprint Objectives

Sprint 1 (March 10 - March 23)

- We want to have the basic features and have a robust skeleton of the game which will be easily extendable.
- We want to have rigorous unit testing for all the components that we implement.
- We want to have code coverage analysis and continuous integration workflows set up for the rest of the project.

Sprint 2 (March 24 - April 6)

- We want to have basic customisation and possibly implement one of the extensions.
- Developing and integrating into a working game.
- We want to begin integration testing for the features that we have implemented and set up a workflow to continuously test new features

Sprint 3 (April 7 – April 17 STePS)

- We want to include the bells and whistles + other extensions that would be viable and effective to be added to the game and also polish up the game for final distribution/release.
- Finalizing the game
- Closer to the deadline we would want to implement a feature freeze and work on perfecting the features that we already have, sorting out any possible issues.

Individual roles and contribution

Week 7 Admin Tasks

Task	Assignee	Status
Create GitHub Org	Rubesh	
CI/CD	Rubesh	

Week 8 Todos

Task	Assignee	Status
Entity	Zheng Ze	
Projectiles Entity	Vanessa	
Tower Entity	Vanessa	
Units Entity	Zheng Ze	
Storage	Rubesh	
Grid class	Keith	
Level class	Keith	

Week 9 Todos

Task	Assignee	Status
Game Loop	TBC	
Game Engine	TBC	
Event Queue	TBC	
UI + Game Controller	TBC	
UI + Level Designer Controller	TBC	

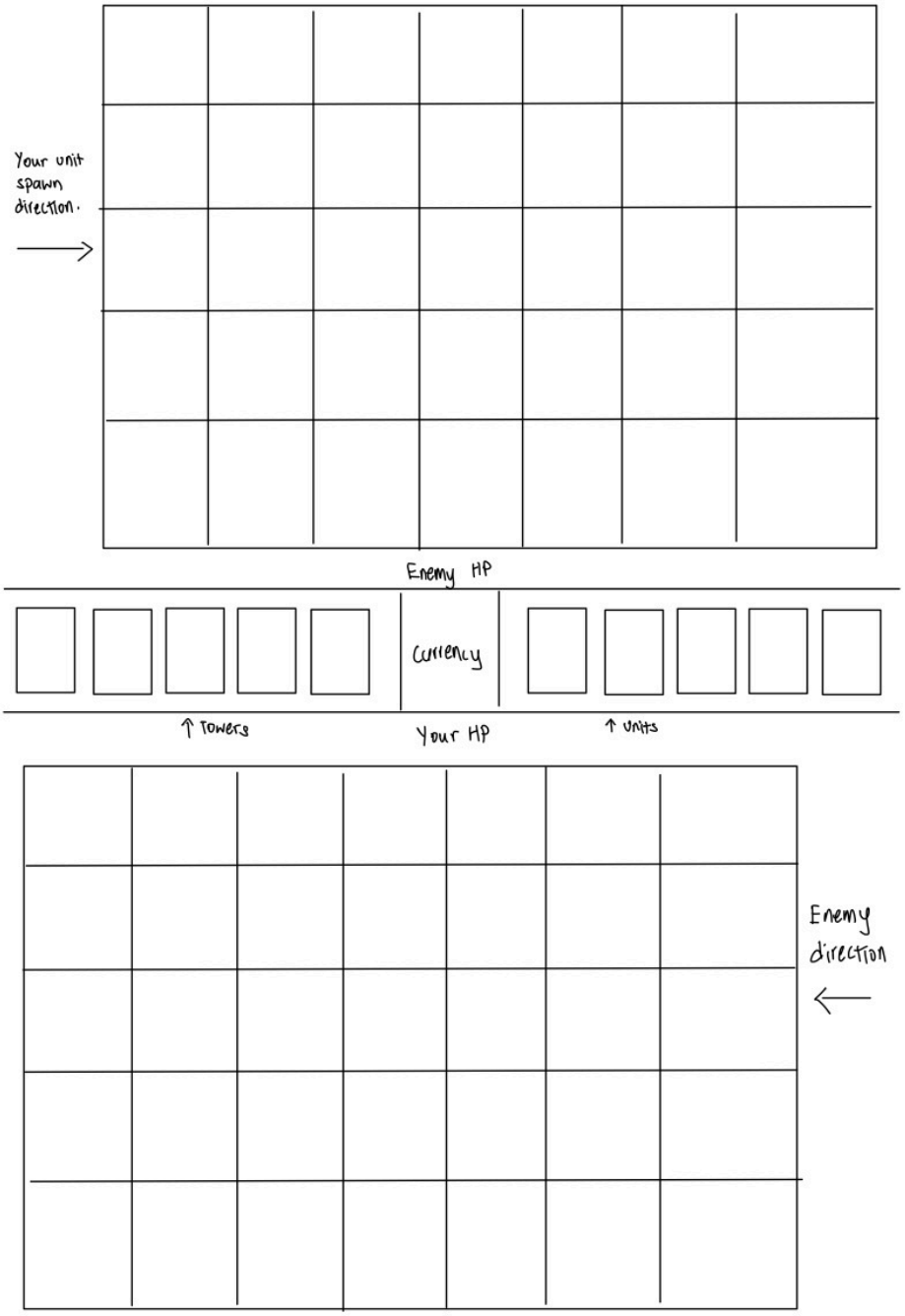
Week 10 Todos

Task (Behaviour Customization)	Assignee	Status
Types	TBC	
Paths	TBC	
Range	TBC	
Enemy Prioritization	TBC	

Diagrams

Mock-ups

The mock-up below is a basic wireframe of what the grid-style level designing will look like.

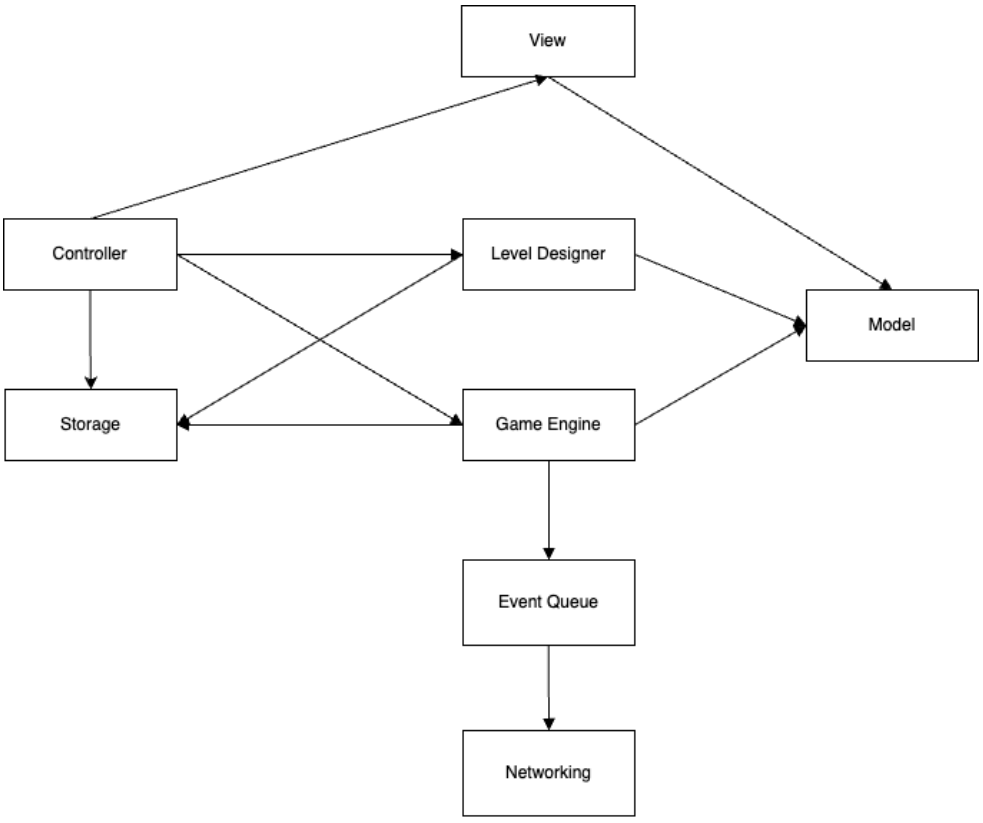


The below image is a basic representation of the game in action, where (amongst other things) dynamic units can be generated and added to the game board.

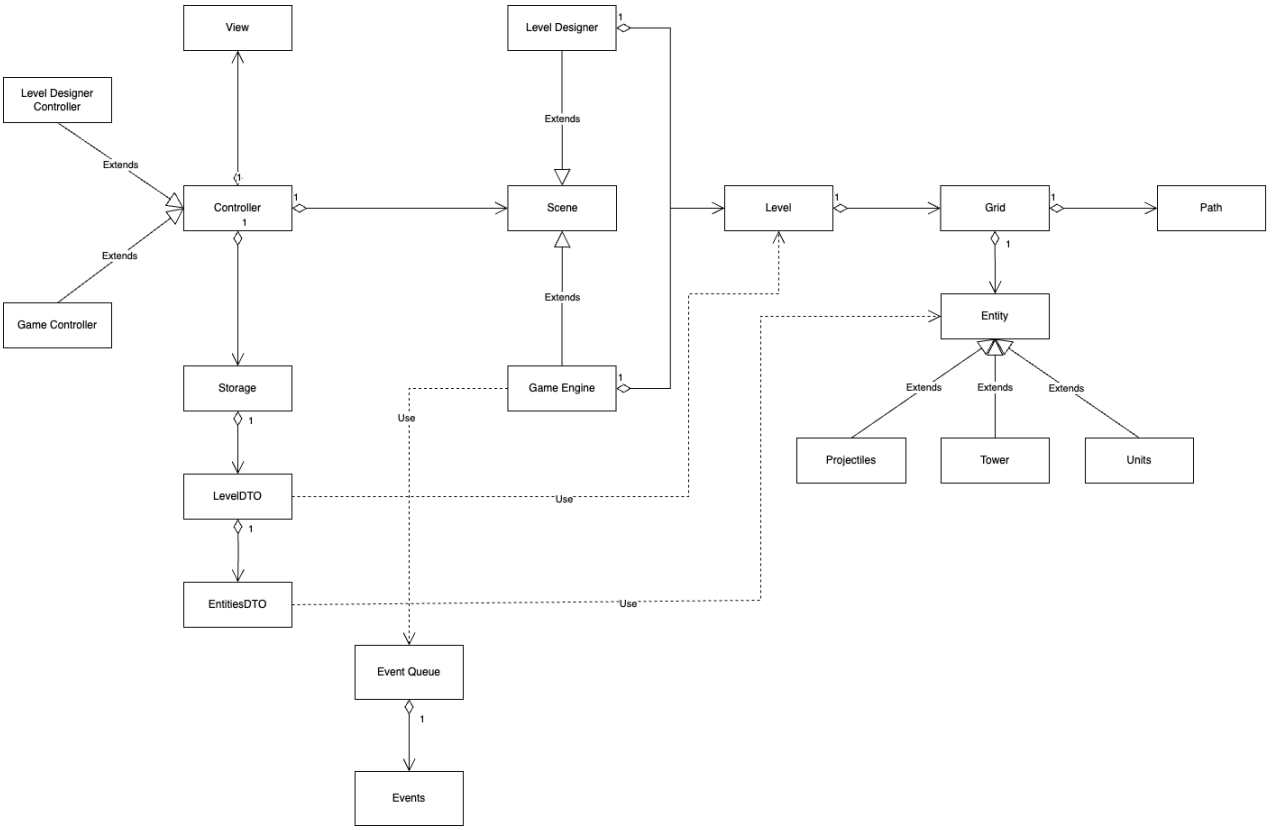


High-Level Design

The below diagram is a high-level architectural overview of the application, following the MVC pattern.



Tentative Class Diagram



Scene / Game Engine / Level Designer
This is responsible for executing the game logic, rules and systems so it is the backbone of the game handling things like game state game events and timing, input processing from the player

Grid
Encapsulates the game area. Entities will be created in one of the spaces of the grid

Path
Represents the direction that the units can go. It lies within the grid.

Event
An action that takes place. For example, a unit was damaged, a unit moves to a new position etc

EventQueue
Data structure that keeps all the events generated during the game. Events are added as a queue so it means that they will occur and be processed subsequently. This ensures that the gameplay events are accurately captured.

Entity
Represents an object in the game world

Storage
Interface that the game can use to save and load data.

DTOs
Holds all data that represents an object in the game. Acts as a translator between the game objects and the storage objects.